

Using the Lottery Ticket Hypothesis to Decrease the Size of LLMs

Author: [Len Washington III](#)  (A20484223)

Artifact:  [lwashington3/cs595-2-project](#)

1. Motivation

Transformers and Large Language Models have allowed for more efficient processing of NLP tasks. One such example where LLMs have been implemented is the EXSCLAIM pipeline,¹ which utilizes LLMs to split captions from research papers to match with any subfigures within the overall (master) figure. However, the biggest problem with running EXSCLAIM on its current servers is that there are no GPUs, which significantly reduces how fast the pipeline can run. Coupled with the dozens of captions that may need to be separated, the Caption Distributor is easily the largest bottleneck within the pipeline. The default LLM (Llama 3.2) can do a fair job of splitting these captions, though it will either need to be replaced by a better model (which would have to be larger) or have some term of improvement done. To address these issues, a method was sought out to increase the inference speed without disrupting the current accuracy of the models. A pruning method known as the Lottery Ticket Hypothesis was chosen, since it had claims of reducing a neural network's weights by 95% while still maintaining similar accuracies as the unpruned version.

2. Background

The Lottery Ticket Hypothesis states that “[any] randomly-initialized, dense neural network contains a subnetwork that is initialized such that—when trained in isolation—it can match the test accuracy of the original network after training for at most the same number of iterations.”²

3. Methodology

To reproduce the Lottery Ticket Hypothesis with a LLM that I did not initially train, nor possess the dataset, several steps are required. The first is two obtain the raw weights of a given LLM, as well as a dataset that could be used to pretrain the weights ... Once that is done, the other metrics are calculated and stored, and the process begins for the next model evaluating for the same row.

Each layer within the model will be subjected to magnitude pruning, where the weights are sorted by magnitude, then the smallest $p\%$ of parameters are removed (or masked out when it comes to the evaluations).

To evaluate the results, a pipeline was constructed to take in the base model (whether it's the original model or pretrained model) and applying the mask found previously. Once the model has been masked, a time is used to time how long the overarching process is, as

well as how long it takes to generate the first token once the inference has begun. After that, the model is called again, this time to receive the logits from the output to calculate the Perplexity of the return, ...

4. Experimental Setup

Table 1: Experimental Specifications

GPU	NVIDIA A100 80GB PCIe
CPU	AMD EPYC 7763 64-Core Processor
Cores	256
RAM	503.414455 GB
Disk Space	418 GB
Chameleon Cloud Site	CHI@UC
Chameleon Cloud Node Name	gigaio-compute-04

The software stack consisted of Python and mainly relied on the HuggingFace suite, namely Transformers, Datasets, and Safetensors. The full list of libraries and versions can be seen in Appendix Section A. The baseline model that was used for pruning and evaluation was [google/gemma-3-1b-it](#) along with [abisee/cnn_dailymail](#) as the dataset. The metrics used to evaluate the effectiveness of pruning were Time-to-First-Token (TTFT), Time per Output Tokens (TPOT), ROGUE-L F1 score, and Perplexity.

5. Results

The experiments found that none of the predictions from the baseline model, nor any of my pruned models (pretrained or not), held up good for what was expected for the dataset. As such, the results will show how well the pruned models did at matching the response of the baseline model.

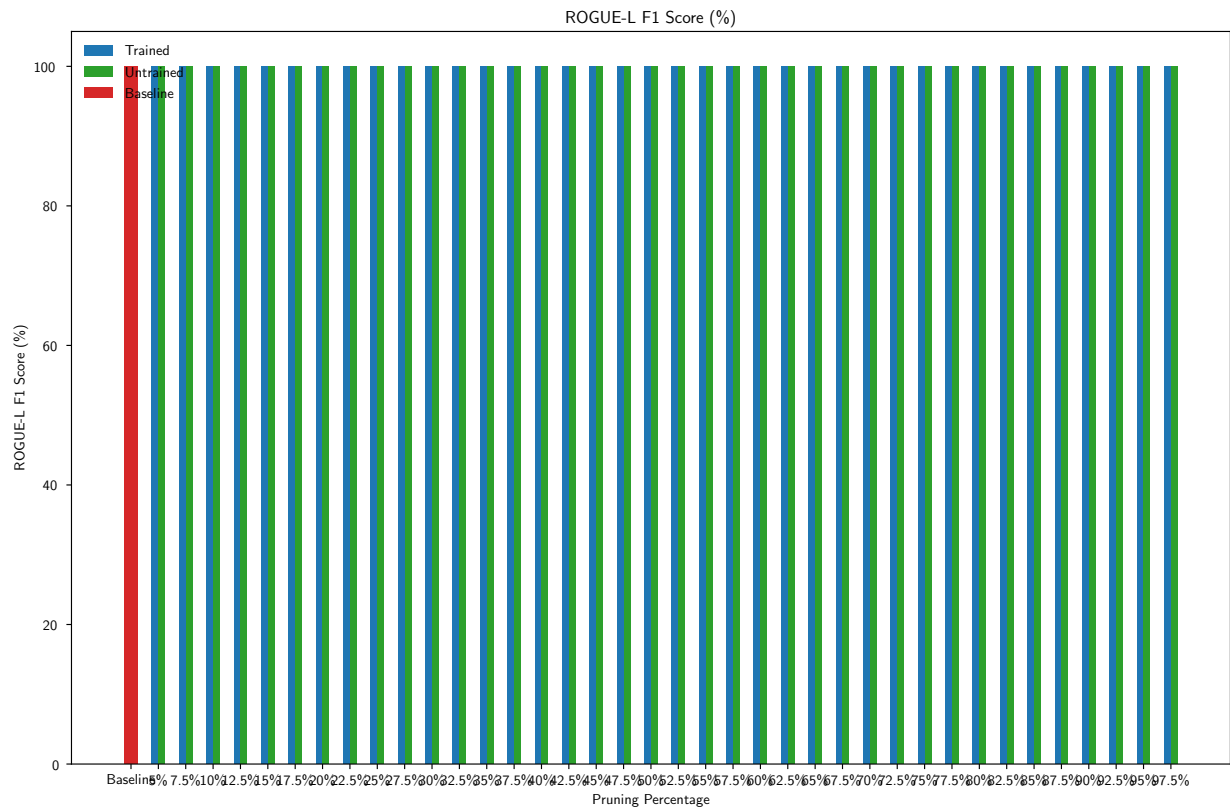


Figure 1: The ROUGE F1 Score for the Longest Common Subsequence.

Table 2: Row 1 (Pre-trained before Pruning).

Trained	Pruning Percentage	Perplexity	ROGUE-L F1 Score (%)	TTFT (ms)	Run Time (ms)	Output Tokens	Time per Output Token (ms)
-	-	32.0966	100.0000%	91.138	464.422	629	0.738
True	5%	501,320.0508	100.0000%	1.097	86.160	629	0.137
True	7.5%	442,413.3920	100.0000%	1.050	85.387	629	0.136
True	10%	776,459.6839	100.0000%	1.056	85.994	629	0.137
True	12.5%	826,537.0311	100.0000%	1.099	86.324	629	0.137
True	15%	826,537.0311	100.0000%	1.065	85.924	629	0.137
True	17.5%	1,544,174.4671	100.0000%	1.106	86.359	629	0.137
True	20%	1,362,729.1843	100.0000%	1.141	86.651	629	0.138
True	22.5%	2,884,897.7057	100.0000%	1.066	85.477	629	0.136
True	25%	3,479,850.8791	100.0000%	1.064	85.802	629	0.136
True	27.5%	2,110,636.2494	100.0000%	1.108	86.235	629	0.137
True	30%	3,943,187.6406	100.0000%	1.112	86.618	629	0.138
True	32.5%	3,943,187.6406	100.0000%	1.106	86.329	629	0.137
True	35%	6,501,217.3374	100.0000%	1.054	85.786	629	0.136
True	37.5%	10,069,282.3901	100.0000%	1.074	86.707	629	0.138
True	40%	8,347,728.3006	100.0000%	1.098	86.516	629	0.138
True	42.5%	7,841,965.0104	100.0000%	1.057	86.300	629	0.137
True	45%	11,409,991.7638	100.0000%	1.129	91.458	629	0.145
True	47.5%	6,920,509.8318	100.0000%	1.102	86.070	629	0.137
True	50%	14,650,719.4290	100.0000%	1.066	85.938	629	0.137
True	52.5%	8,886,110.5205	100.0000%	1.114	86.358	629	0.137
True	55%	14,650,719.4290	100.0000%	1.107	86.379	629	0.137
True	57.5%	24,154,952.7536	100.0000%	1.107	86.143	629	0.137
True	60%	21,316,670.9871	100.0000%	1.107	86.248	629	0.137
True	62.5%	14,650,719.4290	100.0000%	1.093	86.256	629	0.137

Trained	Pruning Percent-age	Perplexity	ROGUE-L F1 Score (%)	TTFT (ms)	Run Time (ms)	Output Tokens	Time per Output Token (ms)
-	-	32.0966	100.0000%	91.138	464.422	629	0.738
False	5%	323,676.5520	100.0000%	1.072	88.155	629	0.140
False	7.5%	344,551.8961	100.0000%	1.056	88.073	629	0.140
False	10%	366,773.5842	100.0000%	1.056	88.463	629	0.141
False	12.5%	268,337.2865	100.0000%	1.068	88.478	629	0.141
False	15%	323,676.5520	100.0000%	1.061	88.483	629	0.141
False	17.5%	323,676.5520	100.0000%	1.072	88.334	629	0.140
False	20%	344,551.8961	100.0000%	1.070	88.569	629	0.141
False	22.5%	323,676.5520	100.0000%	1.075	89.893	629	0.143
False	25%	323,676.5520	100.0000%	1.067	90.256	629	0.143
False	27.5%	323,676.5520	100.0000%	1.070	90.710	629	0.144
False	30%	344,551.8961	100.0000%	1.086	89.606	629	0.142
False	32.5%	304,065.9811	100.0000%	1.085	90.918	629	0.145
False	35%	344,551.8961	100.0000%	1.136	91.268	629	0.145
False	37.5%	366,773.5842	100.0000%	1.099	91.295	629	0.145
False	40%	323,676.5520	100.0000%	1.082	91.013	629	0.145
False	42.5%	344,551.8961	100.0000%	1.085	90.752	629	0.144
False	45%	344,551.8961	100.0000%	1.084	90.929	629	0.145
False	47.5%	323,676.5520	100.0000%	1.074	90.947	629	0.145
False	50%	304,065.9811	100.0000%	1.085	90.402	629	0.144
False	52.5%	344,551.8961	100.0000%	1.103	88.853	629	0.141
False	55%	304,065.9811	100.0000%	1.110	90.711	629	0.144
False	57.5%	304,065.9811	100.0000%	1.102	90.898	629	0.145
False	60%	323,676.5520	100.0000%	1.082	91.549	629	0.146
False	62.5%	323,676.5520	100.0000%	1.076	88.590	629	0.141

6. Profiling Analysis

- Reading in models for the evaluation pipeline takes ~ 40 seconds, which can lead to massive wait times if 77 models are being evaluated (leading to a wait time of about 51 minutes before the evaluation actually starts)

7. Reproducibility

The artifact can be found at <https://github.com/lwashington3/cs595-2-project>

8. Limitations

- The only a few values from the pretraining set could be used for training before the GPU's memory was full.

9. Future Works

- Attempt to figure out the maximum size of the pretraining set that it doesn't go beyond the memory bounds of a device
- Structurally remove transformer layers from the model and test how that affects it
- Figure out a better solution towards saving values (I could save the masked pretrained value in a single file instead of having it distributed [this is only done because I don't want the same version of different files constantly written in different places]), but I not all values need to be written (save space)

10. References

- [1] Eric Schwenker et al. *EXSCLAIM! – An automated pipeline for the construction of labeled materials imaging datasets from literature*. 2021. arXiv: [2103.10631](https://arxiv.org/abs/2103.10631) [cs.IR]. URL: <https://arxiv.org/abs/2103.10631>.
- [2] Jonathan Frankle and Michael Carbin. “The Lottery Ticket Hypothesis: Training Pruned Neural Networks”. In: *CoRR* abs/1803.03635 (2018). arXiv: [1803.03635](https://arxiv.org/abs/1803.03635). URL: <https://arxiv.org/abs/1803.03635>.

Appendix

A. Python Libraries

```
numpy==2.4.4  
matplotlib==3.10.9
```

```
torch==2.11.0
datasets==4.8.5
transformers==5.7.0
evaluate[template]==0.4.6
safetensors==0.7.0
tqdm==4.67.3
huggingface_hub==1.13.0
accelerate==1.13.0
```